TECNOLÓGICO NACIONAL DE MÉXICO
INSTITUTO TECNOLÓGICO DE TIJUANA

SUBDIRECCIÓN ACADÉMICA
DEPARTAMENTO DE SISTEMAS Y COMPUTACIÓN

SEMESTRE:
Enero-Junio

CARRERA:
Ingeniería en Sistemas Computacionales

MATERIA:
Patrones de Diseño

TÍTULO ACTIVIDAD:
Actividad de valor en equipo - Patrón decorador y bridge

NOMBRE Y NÚMERO DE CONTROL DEL ALUMNO:

Chávez Moreno Roberto 22210292

Serna Segura Noel Alberto 22210354

Orozco Hernandez Brandon 21212577

Castillejo Robles Lennyn Alejandro 22210880

NOMBRE DEL MAESTRO:

Maribel Guerrero Luis

FECHA:

26 marzo 2026

Introducción

El Dashboard Estudiantil es una aplicación integral de gestión académica desarrollada para el Instituto Tecnológico de Tijuana. Se trata de un sistema que implementa los patrones de diseño Decorador y Bridge para crear una arquitectura modular, flexible y extensible que permite monitorear métricas académicas, gestionar información de estudiantes y analizar factores de riesgo educativo.

El patrón Decorador se aplica a las gráficas y visualizaciones, permitiendo que los usuarios personalicen dinámicamente aspectos como colores, temas, tamaños y estilos sin modificar el código base de los componentes gráficos. El patrón Bridge se implementa en el sistema de notificaciones y distribución de reportes, desvinculando la lógica de envío abstracta de sus implementaciones concretas (WhatsApp y Gmail), permitiendo agregar fácilmente nuevos canales de comunicación.

La aplicación proporciona a los docentes y administradores académicos una plataforma centralizada para visualizar datos de rendimiento estudiantil en tiempo real, identificar estudiantes en situación de riesgo y compartir información académica a través de múltiples canales. El sistema utiliza tecnologías modernas como Vue.js 3 en el frontend y Node.js con Express en el backend, garantizando una experiencia de usuario responsiva y un backend robusto y escalable.

Objetivo General

Desarrollar un sistema de dashboard académico con arquitectura flexible basada en los patrones de diseño Decorador y Bridge que permita gestionar, analizar, visualizar y distribuir información académica de estudiantes de forma integral, personalizable y eficiente.

Objetivos Específicos

1. Implementar el Patrón Decorador en Gráficas

- Permitir personalización dinámica de visualizaciones gráficas
- Aplicar decoradores para modificar colores, temas, estilos y tamaños de gráficas
- Agregar funcionalidades de visualización sin alterar componentes base
- Crear una biblioteca de decoradores reutilizables para gráficas

2. Implementar el Patrón Bridge en Sistema de Distribución

- Desvincular presentación de datos de sus canales de envío
- Crear puntes entre reportes académicos y múltiples plataformas de mensajería

- Implementar envío de reportes por WhatsApp y Gmail
- Facilitar la adición de nuevos canales de distribución (SMS, Telegram, etc.)

3. Proporcionar Análisis Académico Interactivo

- Presentar métricas en tiempo real (total de estudiantes, tasas de aprobación/reprobación)
- Generar visualizaciones gráficas personalizables de datos académicos
- Crear reportes detallados de desempeño estudiantil
- Permitir configuración visual según preferencias del usuario

4. Identificar Factores de Riesgo

- Detectar automáticamente estudiantes en situación de riesgo académico
- Analizar métricas de riesgo (asistencia, calificaciones)
- Generar alertas tempranas para intervención docente
- Distribuir reportes de riesgo por WhatsApp o Gmail

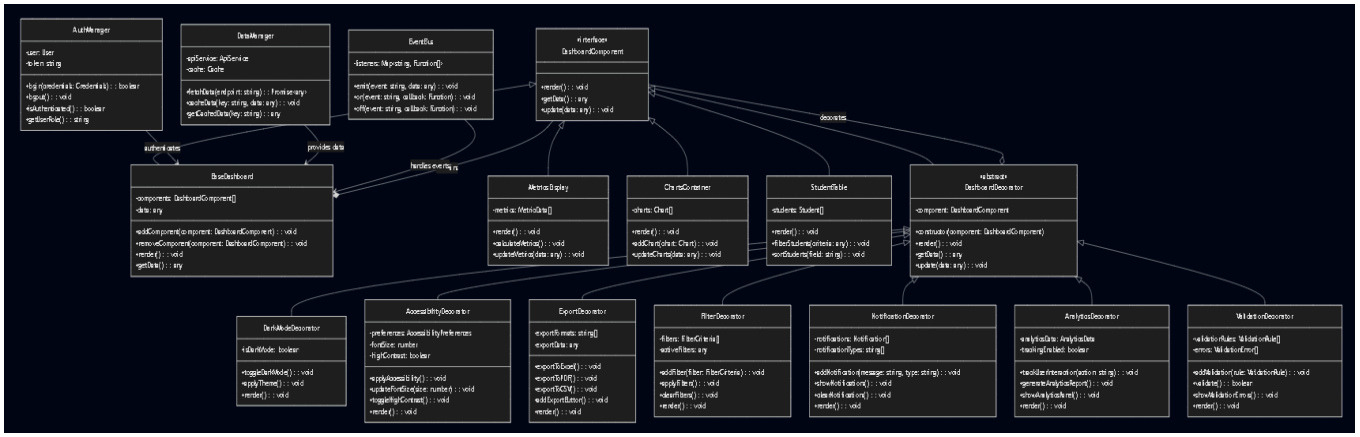
5. Facilitar la Gestión y Distribución de Datos

- Importar/exportar datos en múltiples formatos (Excel, CSV, PDF)
- Validar información de entrada en tiempo real
- Permitir carga masiva de información estudiantil
- Enviar reportes generados a través de WhatsApp y Gmail

6. Mejorar la Experiencia del Usuario

- Proporcionar interfaz intuitiva y amigable
- Permitir personalización de gráficas mediante Decorador
- Implementar opciones intuitivas para compartir reportes
- Desarrollar navegación fluida y responsive

Diagrama Uml



Capturas del código

```
export function useChartDecorator(initialData, initialOptions = {}) {
  // Estado reactivo
  const data = ref(initialData || [])
  const options = ref(initialOptions)
  const chartConfig = ref(null)
  const isLoading = ref(false)
  const error = ref(null)

  // Computed properties
  const hasData = computed(() => {
    return data.value && data.value.length > 0
  })

  const dataCount = computed(() => {
    return data.value?.length || 0
  })

  /**
   * Crea una gráfica de pastel con decoradores
   * @param {Object} config - Configuración de la gráfica
   * @returns {Object} Configuración completa de la gráfica
   */
  const createPieChart = (config = {}) => {
    try {
      isLoading.value = true
      error.value = null

      const {
        filters = {},
        animation = 'bounce',
        theme = 'light',
        enableExport = true,
        customOptions = {}
      } = config

      // Crear gráfica base
      let chart = new BaseChart(data.value, { ...options.value, ...customOptions })

      // Aplicar decoradores en orden
      chart = new PieChartDecorator(chart)

      if (Object.keys(filters).length > 0) {
        chart = new FilterDecorator(chart, filters)
      }

      chart = new ThemeDecorator(chart, theme)
      chart = new AnimationDecorator(chart, animation)
```

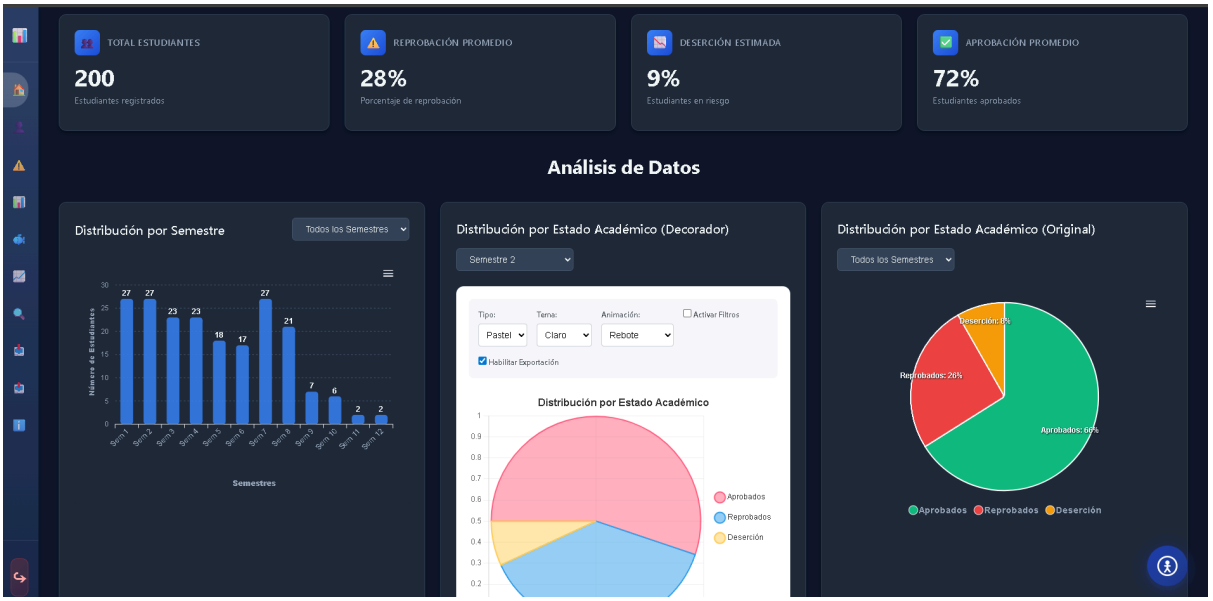
```
 * Crea una gráfica de donut con decoradores
 * @param {Object} config - Configuración de la gráfica
 * @returns {Object} Configuración completa de la gráfica
 */
const createDonutChart = (config = {}) => {
  try {
    isLoading.value = true
    error.value = null

    const {
      filters = {},
      animation = 'bounce',
      theme = 'light',
      enableExport = true,
      cutoutPercentage = 50,
      customOptions = {}
    } = config

    // Crear gráfica base
    let chart = new BaseChart(data.value, { ...options.value, ...customOptions })

    // Aplicar decoradores en orden
    chart = new DonutChartDecorator(chart, cutoutPercentage)

    if (Object.keys(filters).length > 0) {
      chart = new FilterDecorator(chart, filters)
    }

    chart = new ThemeDecorator(chart, theme)
    chart = new AnimationDecorator(chart, animation)

    if (enableExport) {
      chart = new ExportDecorator(chart)
    }

    chartConfig.value = chart.render()
    return chartConfig.value

  } catch (err) {
    error.value = err.message
    console.error('Error creando gráfica de donut:', err)
    return null
  } finally {
    isLoading.value = false
  }
}
```

```
 * Crea una gráfica con decoradores personalizados
 * @param {Array} decorators - Array de funciones decoradoras
 * @param {Object} customOptions - Opciones personalizadas
 * @returns {Object} Configuración completa de la gráfica
 */
const createCustomChart = (decorators, customOptions = {}) => {
  try {
    isLoading.value = true
    error.value = null

    // Crear gráfica base
    let chart = new BaseChart(data.value, { ...options.value, ...customOptions })

    // Aplicar decoradores personalizados
    decorators.forEach(decoratorFn => {
      chart = decoratorFn(chart)
    })

    chartConfig.value = chart.render()
    return chartConfig.value

  } catch (err) {
    error.value = err.message
    console.error('Error creando gráfica personalizada:', err)
    return null
  } finally {
    isLoading.value = false
  }
}

/**
 * Actualiza los datos de la gráfica
 * @param {Array} newData - Nuevos datos
 */
const updateData = (newData) => {
  data.value = newData
}

/**
 * Actualiza las opciones de la gráfica
 * @param {Object} newOptions - Nuevas opciones
 */
const updateOptions = (newOptions) => {
  options.value = { ...options.value, ...newOptions }
}
```

```javascript
/**
 * Abstracción principal del patrón Bridge para Canales de Exportación
 *
 * Esta clase define la interfaz común para todos los canales de exportación
 * permitiendo que la implementación varíe independientemente.
 */
export class ExportChannel {
  constructor(destination) {
    this.destination = destination
  }

  /**
   * Envía datos exportados a través del canal específico
   * @param {Object} exportData - Datos exportados (blob, formato, etc.)
   * @param {Object} options - Opciones específicas del canal
   * @returns {Promise<Object>} Resultado del envío
   */
  async send(exportData, options = {}) {
    if (!this.destination) {
      throw new Error('No se ha configurado un destino para el canal de exportación')
    }

    try {
      return await this.destination.send(exportData, options)
    } catch (error) {
      throw new Error(`Error en el canal de exportación: ${error.message}`)
    }
  }

  /**
   * Obtiene información del canal
   * @returns {Object} Información del canal
   */
  getInfo() {
    return {
      name: this.destination.constructor.name,
      type: this.destination.type || 'unknown',
      supportedFormats: this.destination.supportedFormats || []
    }
  }
}
```

```
export class EmailDestination {
  constructor() {
    this.type = 'email'
    this.supportedFormats = ['png', 'jpg', 'pdf', 'svg']
  }

  async send(exportData, options = {}) {
    const to = options.to
    if (!to) {
      throw new Error('Falta el correo destinatario')
    }

    const filename = options.filename || exportData.filename


    const subject = encodeURIComponent(options.subject || 'Grafica exportada')
    const body = encodeURIComponent(
      `${options.body || 'Se genero una grafica exportada.'}\n\nArchivo: ${filename}\nFormato: ${exportData.format.toUpperCase()}\n\
    )

    window.location.href = `gmail:${encodeURIComponent(to)}?subject=${subject}&body=${body}`

    return {
      success: true,
      channel: 'email',
      to,
      filename,
      format: exportData.format,
      message: 'Se abrio el cliente de correo (adjunta el archivo manualmente)'
    }
  }
}
```

```
1   export class WhatsAppDestination {
2     constructor() {
3       this.type = 'whatsapp'
4       this.supportedFormats = ['png', 'jpg', 'pdf']
5     }
6
7     async send(exportData, options = {}) {
8       const phone = options.phone
9       if (!phone) {
10        throw new Error('Falta el numero de WhatsApp')
11      }
12
13      const filename = options.filename || exportData.filename
14      const message = options.message || 'Te comparto una grafica exportada.'
15
16
17      const text = encodeURIComponent(
18        `${message}\n\nArchivo: ${filename}\nFormato: ${exportData.format.toUpperCase()}\n\
19      )
20
21      const normalizedPhone = String(phone).replace(/\D/g, '')
22      const url = `https://wa.me/${normalizedPhone}?text=${text}`
23      window.open(url, '_blank')
24
25      return {
26        success: true,
27        channel: 'whatsapp',
28        phone: normalizedPhone,
29        filename,
30        format: exportData.format,
31        message: 'Se abrio WhatsApp (envia el archivo manualmente)'
32      }
33    }
34  }
35
```


TOTAL ESTUDIANTES
200
Estudiantes registrados
REPROBACIÓN PROMEDIO
28%
Porcentaje de reprobación
DESERCIÓN ESTIMADA
9%
Estudiantes en riesgo
APROBACIÓN PROMEDIO
72%
Estudiantes aprobados
Análisis de Datos
Distribución por Semestre
Todos los Semestres
Número de Estudiantes
Semestres
Distribución por Estado Académico (Decorador)
Semestre 2
Tipo:
Tema:
Animación:
Activar Filtros
Pastel
Claro
Rebote
Habilitar Exportación
Distribución por Estado Académico
Aprobados
Reprobados
Deserción
Distribución por Estado Académico (Original)
Todos los Semestres
Deserción: 8%
Reprobados: 20%
Aprobados: 68%
Aprobados
Reprobados
Deserción

Tipo:
Pastel
Tema:
Claro
Animación:
Rebote
Activar Filtros
Habilitar Exportación
Distribución por Estado Académico
1
0.9
0.8
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0
0
1
2
Aprobados
Reprobados
Deserción
Exportar Gráfica
Enviar por:
Correo
Correo:
correo@ejemplo.com
Mensaje:
Mensaje opcional
Exportar PNG
Exportar JPG
Exportar PDF
Exportar CSV
Exportar JSON

Conclusiones por integrantes del equipo

Roberto

El sistema usa los patrones Decorador y Bridge para ser más flexible, permitiendo personalizar gráficas y enviarlas por WhatsApp y Gmail. Además, ayuda a identificar estudiantes en riesgo y hace que el sistema sea más fácil de mantener y mejorar.

Brandon

Como conclusión considero que el patrón decorador es bastante efectivo a la hora de querer mostrar poca información o más dependiendo lo que busques, así tú puedes agregarle las funcionalidades sin tener que modificar el código fuente

Noel

El Dashboard Estudiantil muestra cómo los patrones Decorador y Bridge hacen que el sistema sea más fácil de usar y adaptar. Permiten personalizar las gráficas y enviar información por distintos medios sin complicar el funcionamiento.

Lennyn

Este sistema demuestra que usar patrones de diseño facilita agregar nuevas funciones sin afectar lo que ya está hecho. Además, ayuda a mantener todo más organizado y sencillo de mejorar con el tiempo.